

1. Unsupervised Learning in a Digital Payment Platform (eSewa Context)

In large-scale digital payment platforms such as eSewa, millions of transactions are generated every day through activities like mobile recharges, utility bill payments, QR payments, online shopping, and bank transfers. Since user behavior changes frequently and labelled datasets are often limited, unsupervised learning becomes highly valuable for discovering hidden patterns, unusual activities, and customer behavior trends without relying heavily on manually labeled data.

Unsupervised learning techniques help organizations improve personalization, strengthen fraud prevention systems, and make better business decisions using transaction data at scale.

1. Customer Segmentation

Problem Definition in Real-World Context

One major challenge for a platform like eSewa is understanding the behavior of different types of users. Some customers use the platform daily for utility payments, while others may only use it occasionally for mobile top-ups or online shopping. Without proper segmentation, it becomes difficult to design personalized offers, improve customer engagement, or retain valuable users.

The business problem is therefore to group users into meaningful categories based on their transaction patterns and behavioral characteristics.

Proposed Unsupervised Learning Approach

This problem can be effectively solved using clustering techniques. Clustering helps identify groups of users who behave similarly without requiring labeled categories beforehand.

A suitable algorithm for this task is **K-Means clustering** because it is simple, scalable, and works efficiently with large transaction datasets.

The model can use features such as:

- Transaction frequency
- Average transaction amount
- Spending categories
- Time of transactions
- Number of active days per month
- Recent transaction activity

After training, the model may automatically generate customer groups such as:

- High-value active users
- Low-frequency users
- Utility-payment-focused users
- Shopping-oriented users
- Newly active customers

These segments provide valuable business insights that marketing and product teams can use for strategic planning.

Integration into Business Systems

The clustering model can run as a batch process daily or weekly. Once users are assigned cluster IDs, the results can be stored in the company database or analytics warehouse.

These customer segments can then support:

- Personalized recommendations
- Cashback and loyalty campaigns
- Targeted advertisements
- User retention programs
- Premium subscription offers

For example, highly active users may receive loyalty rewards, while inactive users may receive promotional discounts to increase engagement.

The results can also be displayed through internal business dashboards for marketing and analytics teams.

Limitation or Challenge

One limitation of K-Means clustering is that the number of clusters (K) must be predefined before training. Choosing the wrong number of clusters may produce poor segmentation results.

Additionally, K-Means is sensitive to outliers and feature scaling, which means noisy transaction data can affect cluster quality.

2. Fraud Detection

Problem Definition in Real-World Context

Fraud detection is another critical problem in digital payment systems. Fraudulent transactions are usually rare, but they can lead to significant financial losses and reduced user trust. Since fraud patterns continuously evolve, collecting labeled fraud data for supervised learning is often difficult.

The goal is to identify suspicious or abnormal transactions that differ significantly from normal user behavior.

Proposed Unsupervised Learning Approach

This problem can be addressed using anomaly detection techniques. Unlike traditional supervised approaches, anomaly detection models learn normal transaction patterns and identify deviations automatically.

Suitable algorithms include:

- Isolation Forest
- Autoencoders
- DBSCAN

The model may analyze features such as:

- Sudden increase in transaction amount
- Unusual transaction time
- Device or IP address changes
- Rapid transaction frequency
- Geographic location differences

For example, if a user who usually performs small local transactions suddenly initiates multiple high-value international transactions within minutes, the model may flag the activity as suspicious.

Integration into Real-Time Systems

Fraud detection systems are most effective when integrated into real-time transaction pipelines.

Each incoming transaction can be assigned an anomaly score. If the score exceeds a predefined threshold, the system can automatically:

- Trigger OTP verification
- Request additional authentication
- Temporarily pause the transaction
- Send alerts for manual review

Batch processing can also be used to analyze historical transaction patterns and generate fraud investigation reports.

This integration helps improve platform security while minimizing financial risks.

Limitation or Challenge

One major challenge in anomaly detection is the high false positive rate. Sometimes legitimate transactions may appear unusual and get flagged incorrectly, causing inconvenience to users.

Another difficulty is threshold tuning, as fraud behavior changes over time and models require continuous monitoring and updates.

Conclusion

Unsupervised learning plays an important role in modern digital payment platforms where large-scale transaction data is available but labeled data is limited. Techniques such as clustering and anomaly detection help organizations uncover hidden patterns, improve user experience, and strengthen security systems.

Customer segmentation enables businesses to better understand their users and deliver personalized services, while anomaly detection helps identify suspicious activities in real time. Although challenges such as parameter tuning, interpretability, and false positives exist, these methods provide substantial business value when properly integrated into both batch and real-time systems.

2. Techniques to Reduce Overfitting in Deep Learning

Overfitting is one of the most common challenges in deep learning. It occurs when a model performs extremely well on training data but fails to generalize to new unseen data. This usually happens when the model learns noise and unnecessary details from the training set instead of learning meaningful patterns.

To reduce overfitting and improve generalization, several regularization techniques are commonly used.

1. Dropout

Intuition Behind Dropout

Dropout is a regularization technique where some neurons are randomly deactivated during training. Instead of allowing the neural network to rely heavily on specific neurons, dropout forces the model to learn more generalized and distributed features.

This can be thought of as training many smaller neural networks simultaneously and averaging their learning behavior.

Effect on Training and Model Behavior

During each training iteration, a certain percentage of neurons are randomly ignored. For example, a dropout rate of 0.5 means that 50% of neurons are temporarily disabled during training.

As a result:

- The model becomes less dependent on specific neurons
- Overfitting is reduced
- Generalization improves
- The network learns more robust patterns

During testing, all neurons are used normally, but their outputs are scaled appropriately.

Real-World Example

In image classification systems such as handwritten digit recognition or facial recognition, dropout prevents the model from memorizing training images. Instead, the model learns general visual patterns such as edges, shapes, and textures.

Limitation

Dropout may slightly slow down training because of the introduced randomness. It can also require more training epochs before convergence.

2. Early Stopping

Intuition Behind Early Stopping

Early stopping is based on the idea that overfitting often begins after the model has trained for too many epochs. Initially, both training and validation accuracy improve. However, after a certain point, the model starts memorizing training data, causing validation performance to decline.

Early stopping prevents this by stopping training at the optimal moment.

Effect on Training and Model Behavior

During training, validation loss is continuously monitored.

- If validation performance improves, training continues
- If validation loss stops improving for several epochs, training is halted

This ensures that the model maintains strong generalization performance without over-learning training noise.

Real-World Example

In fraud detection systems, early stopping helps models remain adaptable to changing fraud behavior. Instead of memorizing old fraud patterns, the model learns broader behavioral characteristics.

Limitation

Early stopping requires a separate validation dataset, which slightly reduces the amount of data available for training. Improper patience settings may also stop training too early.

3. L2 Regularization (Weight Decay)

Intuition Behind L2 Regularization

L2 regularization discourages the model from assigning extremely large values to weights. Large weights often indicate that the model is relying too heavily on certain features.

By penalizing large weights, the model becomes simpler and more generalized.

The penalty term added to the loss function is:

$$\text{Loss} = \text{Original Loss} + \lambda \sum w^2$$
$$\text{Loss} = \text{Original Loss} + \lambda \sum w^2$$

where λ controls the strength of regularization.

Effect on Training and Model Behavior

L2 regularization:

- Keeps model weights small
- Reduces model complexity
- Prevents excessive memorization
- Encourages smoother decision boundaries

As a result, the model becomes less sensitive to noise in the training data.

Real-World Example

In text classification tasks such as spam detection, L2 regularization prevents the model from depending too heavily on a few specific words. Instead, it learns broader language patterns.

Limitation

If regularization strength is too high, the model may become overly simple and underfit the data.

Conclusion

Overfitting can significantly reduce the effectiveness of deep learning models on unseen data. Techniques such as dropout, early stopping, and L2 regularization help improve model generalization by controlling model complexity and reducing memorization.

In practical applications, multiple regularization techniques are often combined together to achieve the best balance between learning capacity and generalization performance.

3. FCN vs. Autoencoder

Neural networks can be designed for different purposes depending on the problem being solved. A Fully Connected Network (FCN) is mainly used for supervised prediction tasks, while an Autoencoder is designed for unsupervised representation learning and data reconstruction.

Architecture Structure

Fully Connected Network (FCN)

A Fully Connected Network consists of:

- Input layer
- One or more hidden layers
- Output layer

In this architecture, every neuron in one layer is connected to every neuron in the next layer. The structure is flexible and does not need to be symmetric.

FCNs are commonly used for:

- Classification
- Regression
- Prediction tasks

Autoencoder

An Autoencoder has a specialized encoder-decoder structure:

1. Encoder — compresses input into a lower-dimensional representation
2. Latent Space (Bottleneck) — stores compressed information
3. Decoder — reconstructs the original input from compressed features

The architecture is usually symmetric around the bottleneck layer.

Objective and Functionality

FCN

The objective of an FCN is to learn a mapping from inputs to outputs.

Example:

- Input: Customer transaction data
- Output: Fraud / Not Fraud

FCNs are therefore used for supervised learning problems where labeled data is available.

Autoencoder

The objective of an Autoencoder is to reconstruct the input itself.

Example:

- Input: Image
- Output: Reconstructed version of the same image

Autoencoders learn meaningful compressed representations without requiring labeled data.

Input–Output Relationship

FCN

In a Fully Connected Network:

- Input and output are different
- The model predicts labels or values

Example:

- Email text → Spam or Not Spam

Autoencoder

In an Autoencoder:

- Input and output are the same
- The model tries to recreate the original input

Example:

- Original image → Reconstructed image
-

Simple Example

FCN Example

Suppose we want to predict whether an email is spam.

- Input: Email features
- Output: Spam / Not Spam label

The network learns a direct mapping between features and labels.

Autoencoder Example

Suppose we want to compress an image.

- Input: Original image
- Output: Reconstructed image

The Autoencoder learns the most important features required to recreate the image accurately.

Application of Autoencoders

Autoencoders for Fraud Detection

One important application of Autoencoders is anomaly detection in financial systems.

How It Works

The Autoencoder is trained using only normal transaction data.

It learns to reconstruct normal behavior patterns with very low reconstruction error.

When fraudulent or unusual transactions are given as input:

- The reconstruction becomes poor
- Reconstruction error increases significantly

- Transactions with high error are flagged as anomalies

Why It Is Effective

Autoencoders work well for fraud detection because:

- Fraud cases are rare
- Labeled fraud data is limited
- The model learns normal behavior automatically
- Unusual patterns become easier to detect

This makes Autoencoders highly useful for large-scale payment platforms and cybersecurity systems.

Conclusion

Fully Connected Networks and Autoencoders serve different purposes in machine learning and deep learning. FCNs are mainly designed for supervised prediction tasks, whereas Autoencoders focus on unsupervised feature learning and data reconstruction.

Autoencoders are especially powerful in applications such as anomaly detection, dimensionality reduction, and image denoising because they can learn compact and meaningful representations from unlabeled data. Their ability to identify abnormal patterns makes them highly valuable in real-world systems such as fraud detection platforms, recommendation systems, and cybersecurity applications.